

尚硅谷嵌入式项目之心电监测仪（寄存器版本）

（作者：尚硅谷研究院）

版本：V1.0.0

第 1 章 项目需求

1.1 项目概述

一个用于监测患者心电信号的嵌入式系统。该系统旨在采集、处理和显示患者的心电数据，以帮助医生和患者跟踪心脏健康状态。系统将使用嵌入式技术，包括传感器、微控制器和用户界面，以实现这一目标。

1.2 功能描述

1.2.1 数据采集

系统需要能够连接到心电传感器，采集患者的心电信号数据。传感器能够捕获心电波形并将数据传输到嵌入式系统。

1.2.2 数据处理

采集的心电数据在嵌入式系统中进行实时处理。

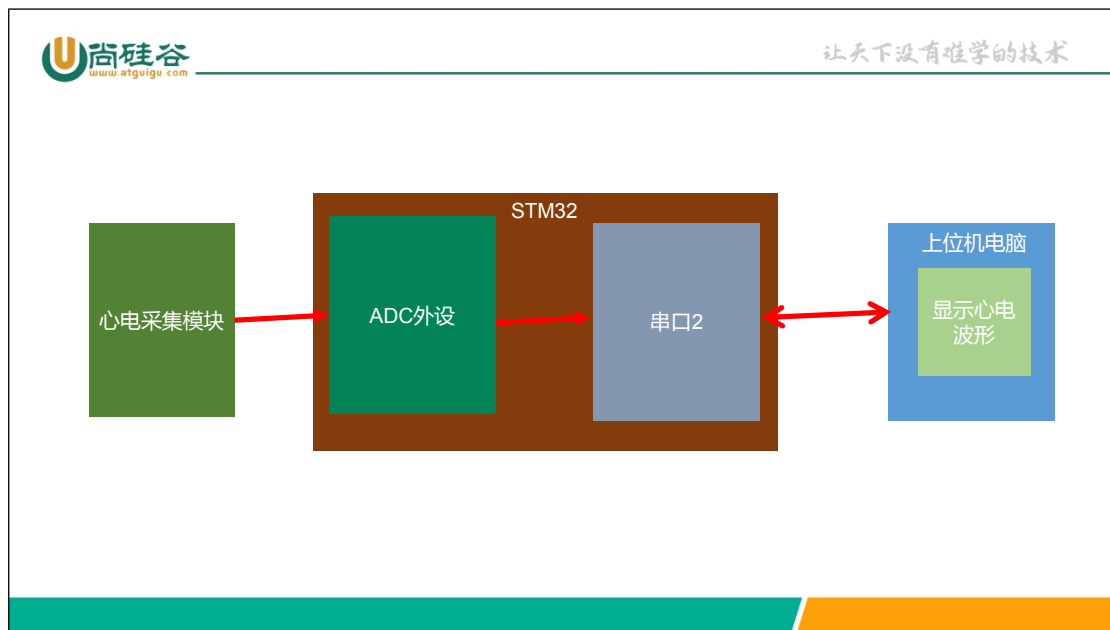
1.2.3 数据显示

具有用户界面，可以实时显示心电波形。

1.2.4 用户交互

用户通过下发采样率和采样时长来控制系统开启采集。

1.3 系统总体设计



第 2 章 硬件架构

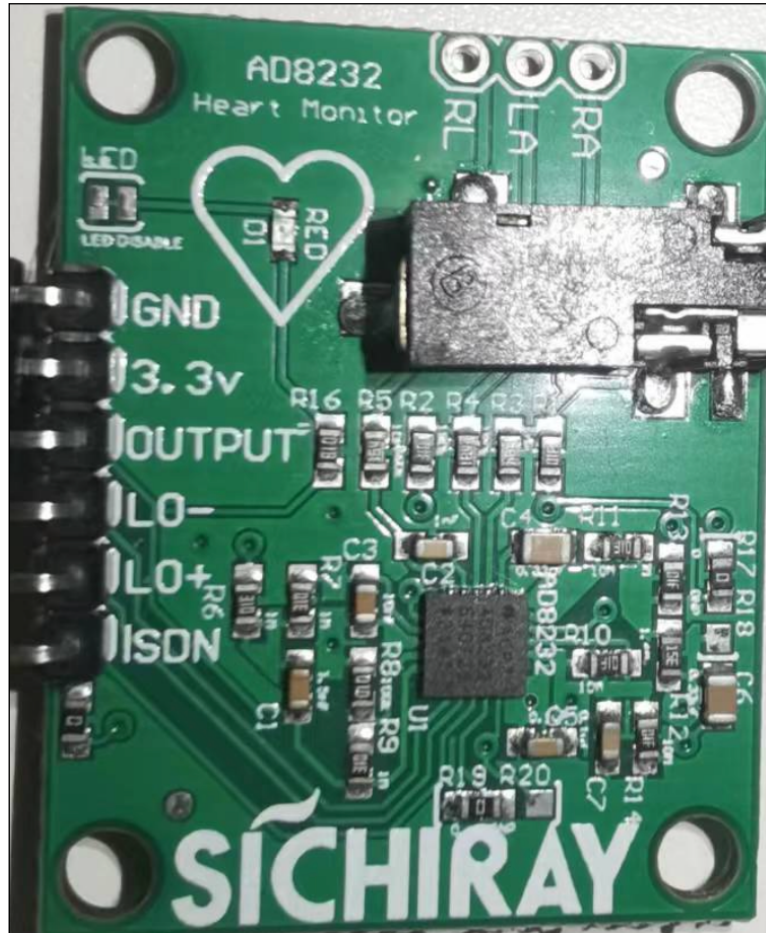
2.1 硬件选型

2.1.1 传感器选型

需要能够采集心电信号并放大心电信号的传感器。目前业界性价比比较高的是 TI 公司的 AD8232。



芯片如果要正常工作还需要一些外围电路，传感器+外围电路就构成了**心电采集模块**。



2.1.2 ADC 选型

ADC 我们选用 STM32 内置 ADC1，使用其 12 通道来接收采集模块放大后的模拟信号。

2.1.3 通讯选型

经过 ADC 转换后的心电数字信号，需要发给上位机进行显示。考虑到数据并不大，我们采用常见的串口通讯。在本项目中我们选用的是串口 2。（串口 1 用于 debug 输出）

2.2 心电采集模块

2.2.1 AD8232 简介

AD8232 是一款生物信号放大器，通常用于心电图（ECG）信号的采集和放大。

2.2.1.1 特点

（1）心电信号放大

AD8232 是专门设计用于放大心电信号的放大器。它可以将微弱的生物信号从人体电极捕获并放大，以便进一步处理和分析。

（2）低噪声

AD8232 具有低噪声放大器，可确保准确的信号放大，减少了环境噪声的影响。

（3）集成电极检测

它集成了用于检测皮肤电极状态的电极检测电路，以确保良好的电极贴合和信号质量。

（4）单电源操作

可以使用单电源供电，降低了电源电路的复杂性。

2.2.1.2 应用领域

（1）医疗设备

AD8232 通常用于医疗设备，如便携式心电监测设备、医用心电图仪器和远程健康监测系统。它可以帮助医生监测患者的心电活动，诊断心脏疾病和异常。

（2）生物传感器

该芯片也可用于开发生物传感器，用于心率监测、生物反馈和健康追踪应用。

（3）研究和教育

AD8232 也可用于学术研究、生物医学工程和教育领域，以帮助学生和研究人员理解心电信号的采集和分析。

总的来说，AD8232 是一款功能强大且灵活的生物信号放大器，适用于各种心电监测和生物传感应用。它的设计使其易于集成到不同类型的电子设备中，以帮助实现生物医疗和生物监测领域的创新。

2.2.2 导联线

在心电监测中，导联线是用于连接**心电电极**与**心电监测仪器**的电缆线。不同的心电监测系统和标准可能会使用不同类型的导联，但一般来说，有以下几种主要的导联类型：

（1）单导联（单极导联）

这是最简单的导联类型，通常用于基本的心电监测。它只使用一个电极，将心电信号相对于身体地线（通常是右腿）测量。这种导联通常用于检测整体的心电活动，而不是特定的心脏区域。

（2）三导联（三极导联）

这是常见的心电监测导联类型，使用三个电极。它包括一个电极放在右腿（身体地线），一个放在左腿或左脚（左腿电极），以及一个放在胸部或手臂上的电极。这种导联通常用于标准的心电图（ECG）检查。

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可[百度访问](#)：尚硅谷官网

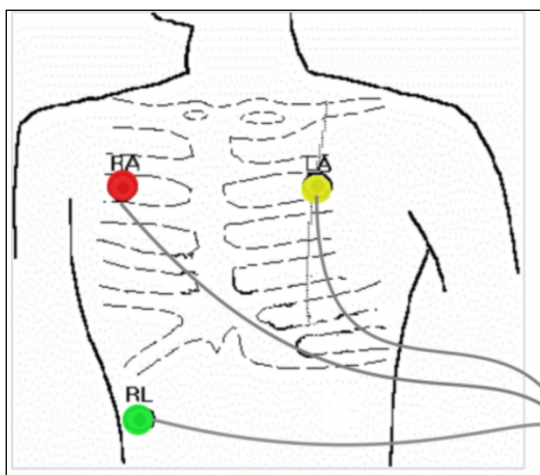
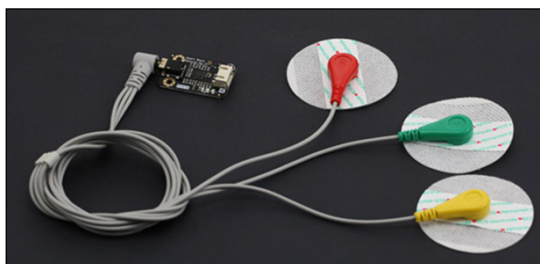
（3）十二导联

这是一种更高级的心电监测导联类型，使用十二个电极，通常包括六个胸部电极和六个四肢电极。它用于详细的心电图检查，可以提供更多心脏区域的信息。

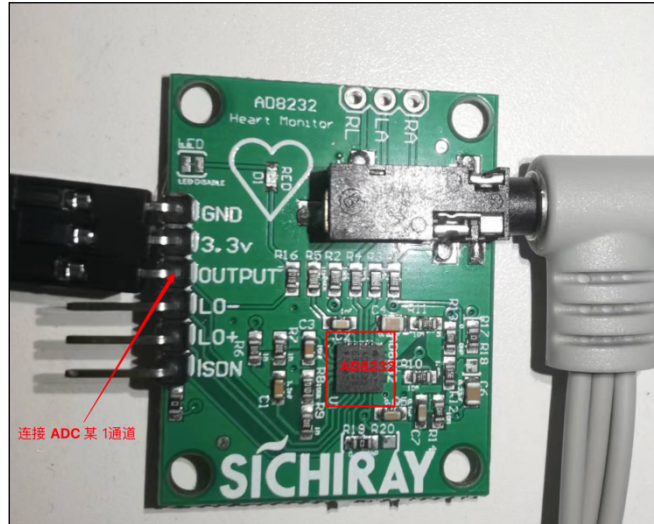
（4）特殊导联

有时候，根据需要，医生或技术人员可以选择使用特殊的导联来监测特定的心脏问题。这可能包括额外的电极或特定的电极布局。

出于成本考虑，我们项目使用的是单导联。



2.2.3 AD8232 引脚接线



GND 和 3.3V 分别接地和电源。OUTPUT 出来的就是经过放大后的心电**模拟信号**，需要连接到 STM32 的 ADC 转成数字信号才能被处理。我们项目中是连接到 **ADC1 的 12 通道**（PC1）。

第 3 章 软件架构

3.1 通用项目软件分层架构

对于大型项目，一般采用分层架构。比较常用的有一种 4 层架构。各层之间一般是上层调用下层，下层不能调用上层。而且同层之间不互相调用。



由于我们这个项目功能比较单一，只划分了**硬件驱动层和应用层**。

3.2 心电监测项目各层模块

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)



3.2.1 硬件驱动层

根据用到的硬件外设来实现需要的驱动，包括：ADC 驱动（ACD1），USART 驱动（USART1 和 USART2），Timer 驱动（TIM6）。

ADC1 用于把采集到的心电模拟信号转成数字信号，然后供 STM32 进行处理。

USART1 用于 Debug，把调试信息发给上位机。

USART2 用于把处理后心电处理信号发送给上位机进行心电波形显示。

TIM6 定时器驱动。使用定时器的更新中断，来控制采样率和采样时长。

3.2.2 应用层

应用层分为 2 个模块：通讯模块和心电采集模块。

通讯模块用于实现与上位机之间的通讯逻辑：解析上位机下发的命令，发送数据到上位机。

心电采集模块用于实现心电采集逻辑和数据处理逻辑。

3.2.3 公共层

我们还开创性的加入了个公共层。比如我们经常调试用的输出 Debug 信息，可能每层都会使用，放在前面的任何一层都不合适，所以才有了公共层。

在这层，该项目提供了两个模块：Debug 模块和延时模块，以方便各层调用。

另外 Debug 的代码只在测试时出现，在 Release 时，通过一个宏控制会去除 Debug 代码。

3.3 硬件的中断服务函数怎么处理

代码分层之后，一般只允许上一层调用下一层代码，不允许下层调用上层和同层调用。但是有时却必须要下层调用上层代码。

比如，硬件的中断服务函数，当中断发生时，我们需要执行一些业务逻辑。中断服务函数理论上属于驱动层，而业务逻辑属于中间层或应用层。这个时候该如何在中断服务函数中执行中间层或应用层的业务逻辑呢？

解决思路 1：不把中断服务函数看作驱动层，在需要的时候，可以把中断服务函数放在任意一层。这种其实是破坏了分层架构，但是实现比较简单。

解决思路 2：使用 **weak 函数**。类似于我们前面学习的 HAL 库实现思路。在中断服务函数中调用 weak 函数，上层在需要的地方重写 weak 函数。我们采用此种方法，简单，而且不破坏分层思想。

解决思路 3：使用 **回调函数** 的思想。上层可以把要实现的业务逻辑通过封装到函数中，把函数指针注册到驱动层，从而实现在底层调用上层代码。这个比较麻烦，而且不好理解，我们暂时不用。

我们在项目代码中使用的是 **第 2 种思路**。

第 4 章 调试模块

4.1 驱动层

4.1.1 Driver_Usart1.h

```
#ifndef __USART1_H
#define __USART1_H

#include "stm32f10x.h"
#include "stdio.h"
void Driver_Usart1_Init(void);
void Driver_Usart1_Start(void);
void Driver_Usart1_stop(void);
void Driver_Usart1_SendChar(uint8_t c);
void Driver_Usart1_SendString(uint8_t *str, uint32_t len);
#endif
```

4.1.2 Driver_Usart1.c

```
#include "Driver_Usart1.h"
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
/* 初始化串口 1 */
void Driver_Uart1_Init(void)
{
    /* 1 开启时钟 1<<14 = 0x4000*/
    /* 1.1 开启 usart1 时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
    /* 1.2 开启 GPIOA 时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    /* 2 配置引脚: PA9(Tx)和 PA10(Rx) */
    /* 2.1 PA9 配置为推挽输出 mode=11 cnf=10*/
    GPIOA->CRH |= (GPIO_CRH_MODE9 | GPIO_CRH_CNF9_1);
    /* 2.2 PA10 配置为浮空输入 */
    GPIOA->CRH |= GPIO_CRH_CNF10_0;

    /* 3 配置波特率为: 115200*/
    USART1->BRR = 0x271;

    /* 4 使能 usart1 的发送和接收 */
    USART1->CR1 |= (USART_CR1_TE | USART_CR1_RE);
}

/**
 * @description: 启动串口 1
 */
void Driver_Uart1_Start(void)
{
    /* 1 使能 usart1 */
    USART1->CR1 |= USART_CR1_UE;
}

/**
 * @description: 停止串口 1
 */
void Driver_Uart1_stop(void)
{
    USART1->CR1 &= ~USART_CR1_UE;
}

/*
 * 向串口 1 发送一个字符
 */
void Driver_Uart1_SendChar(uint8_t c)
```

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
{

    /* 等待发送缓冲区为空。SR_TXE 为 1 表示已经移到移位寄存器， 0 表示还没有 */
    while ((USART1->SR & USART_SR_TXE) == 0)
    {
    }
    /* 把要发送的数据写入到数据寄存器 */
    USART1->DR = c;
}
```

```
/*
    通过串口 1 发送数据：
    参数 1: 要发送的数据
    参数 2: 要发送的数据的长度

*/
void Driver_Uart1_SendString(uint8_t *str, uint32_t len)
{
    uint32_t i;
    for (i = 0; i < len; i++)
    {
        Driver_Uart1_SendChar(str[i]);
    }
}
/*
    printf 函数底层会调用 fputc
    重定向 c 库函数 printf 到串口，重定向后可使用 printf 函数打印数据
*/
int fputc(int ch, FILE *f)
{
    Driver_Uart1_SendChar((uint8_t)ch);
    return (ch);
}
```

4.2 公共层

4.2.1 Debug.h

```
#ifndef __DEBUG_H
#define __DEBUG_H

#include <stdarg.h>
#include "stdio.h"
#include "string.h"
#include "Driver_Uart1.h"
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
/**
 * 为方便调试
 * 当定义 DEBUG 这个宏之后，所有的输出打印语句会正常打印
 * 如果没有定义这个宏，所有的输出打印语句都会在预处理时被去掉。
 * 当调试代码时：    定义 DEBUG
 * 当 Release 代码时： 不定义 DEBUG
 */
#define DEBUG

#ifdef DEBUG
#define debug_init() Debug_Init()
// 定义可变参数版本的 debug_printf
#define FILENAME (strrchr(__FILE__, '\\') ? strrchr(__FILE__, '\\') + 1 : __FILE__)
// 在前面添加文件名和行号。比如要输出字符串"尚硅谷"，则会输出 "[main.c:30]--尚硅谷"
#define debug_printf(format, ...) printf("[%s:%d]--" format, FILENAME, __LINE__,
##__VA_ARGS__)
// 在字符串末尾添加\r\n之后再输出
#define debug_printfln(format, ...) printf("[%s:%d]--" format "\r\n", __FILE__,
__LINE__, ##__VA_ARGS__)

#else
#define debug_init() // Empty definition for RELEASE mode
#define debug_printf(format, ...) // Empty definition for RELEASE mode
#define debug_printfln(format, ...) // Empty definition for RELEASE mode
#endif

void Debug_Init(void);
#endif
```

4.2.2 Debug.c

```
#include "Debug.h"

void Debug_Init(void)
{
    Driver_Usart1_Init();
    Driver_Usart1_Start();
}
```

第 5 章 通讯模块

5.1 驱动层

5.1.1 Driver_Usart2.h

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
#ifndef __USART2_H
#define __USART2_H

#include "stm32f10x.h"

void Driver_USART2_Init(void);
void Driver_USART2_Start(void);
void Driver_USART2_Stop(void);
uint8_t Driver_USART2_ReceiveChar(void);
void Driver_USART2_SendChar(uint8_t c);
void Driver_USART2_SendString(uint8_t *str, uint32_t len);

#endif
```

5.1.2 Driver_Usart2.c

```
#include "Driver_Usart2.h"

/**
 * 串口 2 驱动：
 * 1. 波特率为 57600
 * 2. 开启了接收中断。如果需要在中断中实现业务逻辑请覆写函数：Usart2_RxCallBack
 */

/**
 * @description: 初始化串口 2
 */
void Driver_USART2_Init(void)
{
    /* 1 开启时钟 */
    /* 1.1 开启 USART2 时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
    /* 1.2 开启 GPIOA 时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    /* 2 配置引脚： PA2(Tx)和 PA3(Rx) */
    /* 2.1 PA2 配置为推挽输出 mode=11 cnf=10*/
    GPIOA->CRL |= (GPIO_CRL_MODE2 | GPIO_CRL_CNF2_1);
    /* 2.2 PA3 配置为浮空输入 */
    GPIOA->CRL |= GPIO_CRL_CNF3_0;

    /* 3 配置波特率为： 57600 BRR=39.0621 APB1=36MHz*/
    USART2->BRR = 0x271;

    /* 4 使能 USART2 的发送和接收 */
}
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
USART2->CR1 |= (USART_CR1_TE | USART_CR1_RE);

/* 5 使能 usart2 接收中断 */
USART2->CR1 |= USART_CR1_RXNEIE;

/* 6. 使能空闲中断 */
USART2->CR1 |= USART_CR1_IDLEIE; /* 空闲中断使能 */

/* 7. 设置 NVIC */
NVIC_SetPriorityGrouping(4);
NVIC_SetPriority(USART2_IRQn, 2);
NVIC_EnableIRQ(USART2_IRQn);
}

/**
 * @description: 启动串口 2
 */
void Driver_USART2_Start(void)
{
    /* 5 使能 USART2 */
    USART2->CR1 |= USART_CR1_UE;
}

/**
 * @description: 停止串口 1
 */
void Driver_USART2_Stop(void)
{
    USART2->CR1 &= ~USART_CR1_UE;
}

/**
 * @description: 从串口 2 接收一个字符
 * @return {uint8_t} 接收到的字符
 */
uint8_t Driver_USART2_ReceiveChar(void)
{
    /* 等待接收缓冲区 非空 */
    while ((USART2->SR & USART_SR_RXNE) == 0)
    {
    }
    return USART2->DR;
}
```

```
/**
 * @description: 向串口 2 发送一个字符
 * @param {uint8_t} c 要发送的字符
 */
void Driver_USART2_SendChar(uint8_t c)
{
    /* 等待发送缓冲区为空。SR_TXE 为 1 表示已经移到移位寄存器， 0 表示还没有 */
    while ((USART2->SR & USART_SR_TXE) == 0)
    {
    }
    /* 把要发送的数据写入到数据寄存器 */
    USART2->DR = c;
}

/**
 * @description: 通过串口 2 发送一个字符串
 * @param {uint8_t} *str 要发送的字符串
 * @param {uint32_t} len 要发送的字符串的长度
 */
void Driver_USART2_SendString(uint8_t *str, uint32_t len)
{
    uint32_t i;
    for (i = 0; i < len; i++)
    {
        Driver_USART2_SendChar(str[i]);
    }
}

/**
 * @description: 串口 2 接收回调函数。当接收完一组定长数据后回调这个函数。
 * 使用者应该覆写这个函数。
 * @return {*}
 */
__weak void USART2_RxCallBack(uint8_t rxBuff[], uint16_t count)
{
}

uint8_t rxBuff[100] = {0};
uint16_t count = 0;
#include "Debug.h"

/**
 * @description: 串口 2 中断服务方式
 */
void USART2_IRQHandler(void)
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
{
    if (USART2->SR & USART_SR_RXNE) // 接收到一个字节
    {
        uint8_t c = USART2->DR & 0xff;
        rxBuff[count] = c; // 把数据存入到缓冲区
        count++;           /* 计数加 1 */
    }
    else if (USART2->SR & USART_SR_IDLE) // 产生了空闲中断
    {
        /* 先清除空闲中断位：先读 SR，再读 DR */
        USART2->SR;
        USART2->DR;
        USART2_RxCallback(rxBuff, count);
        count = 0; /* 计数清除，用于下一个变长数据的接收 */
    }
}
```

5.2 应用层

5.2.1 APP_Commucation.h

```
#ifndef __COMMUCATION_H
#define __COMMUCATION_H

#include "Driver_Usart2.h"
#include "stdlib.h"
void APP_Commucation_Start(void);
void APP_Commucation_Stop(void);

void APP_Commucation_CommandProcess(uint16_t *rate, uint16_t *duration);

void APP_Commucation_SendDataToUpperComputer(uint16_t data);

#endif
```

5.2.2 APP_Commucation.c

```
#include "APP_Commucation.h"
#include "Debug.h"
#include "string.h"

/* 接收采样率 */
uint8_t g_receivedRate[10] = {0};

/* 接收采样时长 */
uint8_t g_receivedDuration[10] = {0};
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
/* 接收是否完成。当采样时长接收完毕之后，任务命令接收结束*/
uint8_t g_isReveiveCompleted = 0;

/**
 * @description: 串口 2 接收回调函数
 * 接收上位机传递过来指令。 一共两种指
 * 1. "s1000\n" s 表示下发的是采样率 1000 表示采样率（每 s 采样 1000 次）
 * 2. "c60\n" c 表示下发的是采集时长 60 表示采样 60s。 收到这个命令之后开启 ADC 和定
    时器开始采集
 * 用 \n 表示命令结束
 * @param {uint8_t} c
 * @return {*}
 */
void USART2_RxCallBack(uint8_t rxBuff[], uint16_t count)
{
    if (rxBuff[0] == 's') // 采样率接收完毕
    {
        memcpy(g_receivedRate, &rxBuff[1], count - 2); // 去掉了 s 字符和\n 字符
    }
    else if (rxBuff[0] == 'c') // 采样时长接收完毕
    {
        memcpy(g_receivedDuration, &rxBuff[1], count - 2);
        g_isReveiveCompleted = 1;
    }
}

/**
 * @description:
 */

/**
 * @description: 下位机(单片机)与上位机(电脑)开启通讯
 */
void APP_Commucation_Start(void)
{
    Driver_USART2_Init();
    Driver_USART2_Start();
}

/**
 * @description: 下位机(单片机)与上位机(电脑)停止通讯
 * @return {*}
 */
void APP_Commucation_Stop(void)
```

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
{
    Driver_USART2_Stop();
}

/**
 * @description: 对收到的命令进行处理
 * @param {uint16_t} *rate 采样率
 * @param {uint16_t} *duration 采样时长
 */
void APP_Commucation_CommandProcess(uint16_t *rate, uint16_t *duration)
{
    /* 1. 等待命令接收完毕 */
    while (!g_isReveiveCompleted)
    {
    }
    /*
        2. 把字符形式的采样率和采样时长转换成数字。
        从串口接收的采样率和采样时长是按照字符存储在数组中 比如: ['1','0','0']
        需要把字符转换成 uint16_t 类型, 方便后续处理.
    */
    *rate = atoi((char *)g_receivedRate);
    *duration = atoi((char *)g_receivedDuration);
}

/**
 * @description: 给上位机发送数据
 *
 * 上位机接收的是 4 位的字符数字加一个\n。
 * 比如 123 应该发送 0123\n
 * 19 应该发送 0019\n
 * @param {uint16_t} data 要发送的数据
 * @return {*}
 */
uint8_t txBuff[5] = {0};
void APP_Commucation_SendDataToUpperComputer(uint16_t data)
{
    sprintf((char *)txBuff, "%04lu\n", data);
    Driver_USART2_SendString(txBuff, 5);
    // debug_printf((char *)txBuff);
}
```

第 6 章 心电采集模块

6.1 驱动层

更多 Java -大数据 -前端 -python 人工智能资料下载, 可百度访问: 尚硅谷官网

6.1.1 ADC 驱动

6.1.1.1 Driver_ADC.h

```
#ifndef __ADC_H
#define __ADC_H

#include "stm32f10x.h"

void Driver_ADC1_Init(void);

void Driver_ADC1_Start(void);

void Driver_ADC1_Stop(void);

double Driver_ADC1_ReadValue(void);

#endif
```

6.1.1.2 Driver_ADC.c

```
#include "Driver_ADC.h"

/**
 * @description: 对 ADC1 进行配置
 * @return {*}
 */
void Driver_ADC1_Init(void)
{
    /* 1. 开启时钟 */
    RCC->APB2ENR |= RCC_APB2ENR_ADC1EN;
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;

    /* 2. 配置 PC1 为模拟输入 CNF:00 MODE:00 */
    GPIOC->CRL &= ~GPIO_CRL_MODE1;
    GPIOC->CRL &= ~GPIO_CRL_CNF1;

    /* 3. 禁用扫描模式（只有一个通道，不需要扫描） */
    ADC1->CR1 &= ~ADC_CR1_SCAN;

    /* 4. 启用连续转换（转换不会停止） */
    ADC1->CR2 |= ADC_CR2_CONT;

    /* 5. 转换结果右对齐 0:右对齐 1:左对齐*/
    ADC1->CR2 &= ~ADC_CR2_ALIGN;

    /* 6. 禁用规则通道外部触发 */
```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
ADC1->CR2 &= ~ADC_CR2_EXTTRIG;

/* 7. 使用软件启动转换 111=软件启动*/
ADC1->CR2 |= ADC_CR2_EXTSEL;

/* 8. 规则转换序列长度（有几个通道需要转换）000:1个 001: 2个 ... */
ADC1->SQR1 &= ~ADC_SQR1_L;

/* 9. 规则转换序列中的第一个转换 通道 11, 就是 SQR3[4:0]=01011 */
ADC1->SQR3 &= ~ADC_SQR3_SQ1; /* [4:0]位置为 0 */
ADC1->SQR3 |= 11 << 0; /* 设置最后 5 位 */

/* 10. 通道 11 采样时间 设置 7.5 个周期=001*/
ADC1->SMPR1 &= ~ADC_SMPR1_SMP11_2;
ADC1->SMPR1 &= ~ADC_SMPR1_SMP11_1;
ADC1->SMPR1 |= ADC_SMPR1_SMP11;

}

/**
 * @description: 启动 ADC1 开始转换
 */
void Driver_ADC1_Start(void)
{
    /* 1. 启动 ADC1 ADON: 0->1 是从断电模式下唤醒*/
    ADC1->CR2 |= ADC_CR2_ADON;

    /* 2. 启用 AD 校准, 校准完成之后会自动清 0*/
    ADC1->CR2 |= ADC_CR2_CAL;

    /* 3. 等待校准完成 */
    while (!(ADC1->CR2 & ADC_CR2_CAL))
        ;

    /* 4. 需要再次启动, ADON: 1->1 才会启动转换 */
    ADC1->CR2 |= ADC_CR2_ADON;

    /* 5. 开始转换 规则通道 */
    ADC1->CR2 |= ADC_CR2_SWSTART;

    /* 6. 等待首次转换完成 */
    while (!(ADC1->SR & ADC_SR_EOC))
        ;
}
```

更多 Java -大数据 -前端 -python 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
}

/**
 * @description: 停止 ADC1 转换
 */
void Driver_ADC1_Stop(void)
{
    /* 1. 停止转换 */
    ADC1->CR2 &= ~ADC_CR2_ADON;
}

/**
 * @description: 读取 ADC 采集到的值。直接访问数据寄存器的值。
 * @return 寄存器的值
 */
double Driver_ADC1_ReadValue(void)
{
    /* 12 位, 范围是[0,4095] */
    return ADC1->DR;
}
```

6.1.2 Timer 驱动

6.1.2.1 Driver_Timer.h

```
#ifndef __DRIVER_TIMER_C
#define __DRIVER_TIMER_C

#include "stm32f10x.h"

void Driver_Timer6_init(void);

void Driver_Timer6_Start(void);

void Driver_Timer6_Stop(void);

#endif
```

6.1.2.2 Driver_Timer.c

```
#include "Driver_Timer.h"
#include "Debug.h"

/**
 * 定时器 6 驱动
 * 1. 每 1ms 产生一次更新中断
 * 2. 如果需要在中断中写业务逻辑, 请覆写函数: TIM6_UpInterruptCallBack
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
*/

/**
 * @description: 定时器 6 的初始化
 */
void Driver_Timer6_init(void)
{
    /* 1. 开启时钟 */
    RCC->APB1ENR |= RCC_APB1ENR_TIM6EN;

    /* 2. 设置预分频值：分频值 7199 表示 7200 分频。分频后频率 10K，周期 0.1ms */
    TIM6->PSC = 7200 - 1;

    /* 3. 设置预加载寄存器的值。 */
    TIM6->ARR = 10 - 1; /* 频率 10K，预加载值是 9 则中断周期是 1ms */

    /* 4. 使能定时器的更新中断 */
    TIM6->DIER |= TIM_DIER_UIE;

    /* 5. 设置中断优先级 高于串口中断优先级 */
    NVIC_SetPriority(TIM6_IRQn, 1);

    /* 6. NVIC 中启用 TIM6 的中断 */
    NVIC_EnableIRQ(TIM6_IRQn);

    /* 7. 防止一启动就进入更新中断 */
    TIM6->EGR |= TIM_EGR_UG;
    TIM6->SR &= ~TIM_SR_UIF;
}

/**
 * @description: 定时器开启计时
 */
void Driver_Timer6_Start(void)
{
    /* 使能计数器 */
    TIM6->CR1 |= TIM_CR1_CEN;
}

/**
 * @description: 定时器关闭
 */
void Driver_Timer6_Stop(void)
{

```

更多 [Java](#) -[大数据](#) -[前端](#) -[python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
/* 失能计数器 */
TIM6->CR1 &= ~TIM_CR1_CEN;
}

/**
 * @description: 定时器 6 更新中断回调函数
 * @return {*}
 */
__weak void TIM6_UpInterruptCallBack()
{
}

/* 定时器 6 的中断服务函数。 每 1ms 执行 1 次 */
void TIM6_IRQHandler(void)
{
    /* 务必清除中断标志位，否则会一直进入中断 */
    TIM6->SR &= ~TIM_SR_UIF;

    /* 调用更新中断回调函数 */
    TIM6_UpInterruptCallBack();
}
```

6.2 应用层

6.2.1 APP_HeartCollect.h

```
#ifndef __APP_HEARTCOLLECT_H
#define __APP_HEARTCOLLECT_H

#include "stm32f10x.h"
#include "Driver_ADC.h"
#include "Driver_Timer.h"

void APP_HeartCollect_Start(uint16_t rate, uint16_t duraion);
void APP_HeartCollect_Stop(void);
uint16_t APP_HeartCollect_ReadHeartData(void);
#endif
```

6.2.2 APP_HeartCollect.c

```
#include "APP_HeartCollect.h"

/* 采样时长 ms */
static uint16_t g_duration;
/* 多少 ms 执行 1 次采样 */
static uint16_t g_collectMs;
/* 保存定时器中断次数 */
static uint16_t g_time6InterruptCount;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)


```
/* 是否要读 ADC 取电压值了 */
uint8_t g_isToReadAdcValue = 0;

/**
 * @description: 开始采集心电图
 * @param {uint16_t} rate 采样率 1 分钟的采样多少次
 * @param {uint16_t} duraionSecond 采集时长 s 用来控制定时器的停止。
 */
void APP_HeartCollect_Start(uint16_t rate, uint16_t duraion)
{
    /* 0. 保存采样周期 (ms) 和 多久执行 1 次采样 */
    g_duration = 1000 * duraion;
    g_collectMs = 60 * 1000 / rate;

    /* 1. 初始化 ADC1 */
    Driver_ADC1_Init();

    /* 2. 启动 ADC 开始采集 */
    Driver_ADC1_Start();

    /* 3. 初始化定时器 6 */
    Driver_Timer6_init();

    /* 4. 启动定时器 6 */
    Driver_Timer6_Start();
}

/**
 * @description: 结束采集心电图
 */
void APP_HeartCollect_Stop(void)
{
    Driver_ADC1_Stop();
}

/* 覆写定时器中断回调函数 */
void TIM6_UpInterruptCallBack()
{
    /* 1. 定时器计时超过了采样时长, 则停止定时器 */
    g_time6InterruptCount++;
    if (g_time6InterruptCount >= g_duration)
    {
        Driver_Timer6_Stop();
    }
}
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
        return;
    }
    /* 2. 判断是否到了应该读取心电数据了 */
    if (g_time6InterruptCount % g_collectMs == 0)
    {
        g_isToReadAdcValue = 1;
    }
}

/**
 * @description: 读取心电数据，其实就是 ADC 的电压值
 * @return {*} 读取到的 ADC 电压值。范围 [0-4095]
 */
uint16_t APP_HeartCollect_ReadHeartData(void)
{
    /* 1. 等待定时器告诉我们该去读 adc 值 */
    while(g_isToReadAdcValue == 0);

    /* 2. 把 g_isToReadAdcValue 设置为 0，用于下次读取 */
    g_isToReadAdcValue = 0;

    /* 3. 返回读取到电压值 */
    return Driver_ADC1_ReadValue();
}
```

第 7 章 main.c

```
#include "stm32f10x.h" // Device header

#include "APP_HeartCollect.h"
#include "APP_Commucation.h"
#include "Debug.h"

extern uint8_t g_toReadAdcValue;
int main(void)
{
    debug_init();

    debug_printf("尚硅谷心电监测项目.....\r\n");

    /* 1. 开启 APP 与上位机的通讯 */
    debug_printf("开启与上位机通讯....\r\n");
    APP_Commucation_Start();
```

```

/* 2. 处理上位机传来的命令,得到采样率和采样时长 */
debug_printf("开始处理上位机下发的命令....\r\n");
uint16_t rate, duration;
APP_Commucation_CommandProcess(&rate, &duration);
debug_printf("rate=%d, duration=%d \r\n", rate, duration);
debug_printf("命令接收完毕....\r\n");

/* 3. 开始采集心电数据 */
APP_HeartCollect_Start(rate, duration);

while (1)
{
    /* 4. 读取心电数据 */
    uint16_t heartData = APP_HeartCollect_ReadHeartData();

    /* 5. 上传心电数据到上位机 */
    APP_Commucation_SendDataToUpperComputer(heartData);
}
}

```

第 8 章 上位机

上位机负责下发命令和显示心电图。用 Python 编写，已经打包成 exe 可以在 windows 下直接使用。



注意：软件启动后，按照图示的 **123 顺序**来设置和启动，否则软件会报错。

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

第 9 章 测试

把连线按照如图连接起来。然后打开尚硅谷心电监测仪上位机软件。

- （1）选择表示 STM32 的串口，设置波特率。然后点击对应设置。
- （2）设置采样率（每分钟采样多少条数据）。然后点击对应的设置。
- （3）设置采样时长（采样多久）。然后点击启动。